WarpDial: A Case for Reconfigurability and Explainability in Object Storage

Tejus Chandrashekar
Independent Researcher
tchandrashekar@acm.org

Abstract—Object storage offers near-infinite capacity with zero operational overhead, but its latency can be a concern for HPC and AI workloads. Managed services hide these low-level decisions behind an SLA without exposing how the system achieves its guarantees. Erasure coding is one such decision, providing both storage efficiency and durability. Giving customers direct control over it is dangerous because a poor configuration can compromise an object store's correctness, availability, or durability.

We build WarpDial, a reconfigurability layer on top of WarpDrive [2], an open-source object store. It exposes static, parametric, and imperative placement policies through one interface, with an explainability layer that previews and reports each policy's cost. We validate WarpDial on a nine-node cluster using DuckDB [3], [4] and Lance [5]. A selective TPC-H query drops from 723 ms to 63 ms (∼11.5×), and Lance row-batch latency drops from 54 ms to 19 ms (up to ∼2.9×, less at higher nprobe), with no change in Recall@10.

I. Introduction

Object stores typically apply one fixed placement and erasure-coding policy across an entire pool. Fusion [1] shows what co-designing that policy with the data's own structure can achieve: aligning erasure-coding stripes with Parquet's column-chunk layout lets a selective query skip reconstructing a chunk it never requested, at a reported ∼1.2% storage overhead. We generalize this beyond Fusion's Parquet-only policy to one interface any operator can implement for any format, demonstrated here on Parquet analytics and Lance vector search.

Letting a developer define a policy like Fusion's themselves means trusting code that decides how an object's bytes are laid out: a wrong or adversarial policy can silently break the guarantees an object store exists to provide. We make the following contributions:

- Interface: one interface (PlacementPolicy/StripePacker) that any operator implements to add a policy, instead of forking the engine.
- Explainability: a dry-run cost preview before a write, and a cost report on every PUT response after one, so a caller knows what a policy costs instead of discovering it blind (Fig. 1).

These policies run today as trusted, compiled-in code, with the same build-test-deploy cycle as the storage engine itself. For future workloads, we plan to remove that cycle: a managed service that runs a tenant's own placement code sandboxed through WebAssembly, with no build, test, or deploy step of its own, as the caller moves from a human reviewing each step toward an agent. To our knowledge, no existing object store exposes this kind of pluggable, cost-reporting placement interface to developers.

II. Methodology

Placement classes differ in who decides location and from what, and in how much placement state the decision leaves behind. A static policy derives placement from the object key, bucket policy, and node set (CRUSH-style hashing): one rule per bucket, no per-object record, costing O(buckets). A parametric policy stores a small per-object record (seed, epoch, layout version), costing O(objects). An imperative policy uses its own code to decide placement from the object's data itself, workload-aware, storing one entry per computable unit, costing O(objects × units per object). Fusion's file-format-aware coding is one imperative policy, hardcoded to Parquet; we generalize it behind the interface introduced above, selected per bucket.

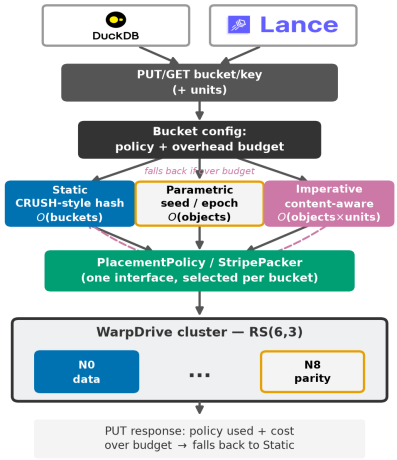


Fig. 1. WarpDial's architecture: three placement classes behind one interface, a per-bucket overhead budget, and a PUT response that reports the policy used and its cost.

We built two concrete imperative policies from an arbitrary $(unit_id, size)$ list and a stripe width k: one groups units by raw size (Fusion's Parquet case or a batch of rows), the other groups first by IVF cluster ID, then by size, for vector-index partitions. Both read computable-unit boundaries from a client-supplied header at PUT time (x-warpd-computable-units: [[offset,len],...]). Fusion's own file-format-aware coding discovers these boundaries itself, parsing a Parquet footer before constructing stripes. We scope our StripePacker to only the second step; generalizing the first into this header lets one interface cover Parquet and Lance's differently structured index without a parser hardcoded to either. A pluggable parser interface instead, built-in or user-defined per format, is future work, kept separate from the packer so one packing algorithm serves many formats and one format serves many algorithms.

A bucket's configuration names its policy and the overhead threshold described above; exceeding it falls back to static, whole-object erasure coding. The fallback itself is Fusion's own mechanism, made per-bucket instead of a single global constant; our own addition is making its outcome knowable rather than silent. A dry-run endpoint runs this calculation against declared units with no body or write, so cost is knowable before committing, not only after. Letting each bucket pick its own (k, m) stripe width also loses the uniform failure-domain and free-space accounting a single pool-wide scheme gives, unmeasured

Workload	stripes touched	bytes reconstructed	overhead
TPC-H Q6 (sel.)	1.0/132	1.6 MB/208.5 MB	0.21%
TPC-H Q1 (low sel.)	2.2/132	3.5 MB/208.5 MB	0.21%
SIFT1M-small, data	1.0/84	62 KB/5.1 MB	1.18%
SIFT1M-small, aux-idx	2.6/37	23 KB/311 KB	412%
Row-batch, data	1.0/167	368 KB/61.5 MB	0.14%
Row-batch, aux-idx	2.6/96	28 KB/992 KB	447%

TABLE I. Reconstruction cost per query, RS(6,3): stripes touched, bytes moved, and overhead, for both the data fragment and IVF aux-index of each Lance workload.

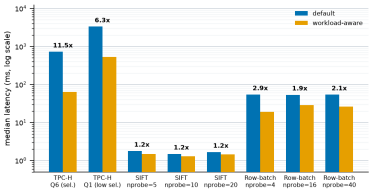


Fig. 2. Median latency, RS(6,3), log scale: default vs. workload-aware placement, across all eight runs.

here and left as the natural next question for running WarpDial across many buckets.

III. Preliminary Evaluation

A. Performance

A nine-node cluster (one process per node) runs RS(6,3) on one machine (Apple M3, 8 cores, 16 GB RAM). We evaluate only the imperative class, since static and parametric placement already exist in other object stores, through two variants: Fusion-style placement for Parquet via DuckDB [3], [4] and a new policy for Lance [5] vector search, each against a static baseline.

We uploaded the dbgen TPC-H lineitem table and a Lance IVF_PQ index over the published SIFT1M-small corpus [6], each once under the static policy and once under its workload-aware imperative policy, verifying byte-identical uploads before timing. We ran TPC-H's Q6 (selective) and Q1 (low-selectivity) against the TPC-H upload (Table I). A separate, larger Lance run (20,000 vectors, 768 dims, 283 partitions) showed that a query spends most of its time reconstructing the ∼60 MB row-data fragment, not the small IVF auxiliary file, so we applied the size-grouped policy to that fragment using 20-row batches.

Table I and Fig. 2 summarize every result. Q6's output matched exactly between buckets, as it must: a workload-aware policy changes how many bytes a query moves, never what it returns, and the same holds for Recall@10 and the row-batch IDs/vectors. The speedup tends to track file size: largest on the 208 MB TPC-H file, smaller on the 60 MB row-data fragment, smallest on SIFT1M-small's much smaller files. The auxiliary index's much higher overhead is expected: a small object's fixed per-stripe padding and parity cannot be amortized over many data bytes, which is why every fragment reported here stays under 1.2%.

B. Safety and explainability of cost

We want every misconfigured or malicious StripePacker rejected before it writes a shard, not silently accepted. A StripePacker never sees or supplies an object's bytes, only which computable unit goes in which bin, how many bins, and a claimed per-stripe capacity: that is the complete surface it can lie about.

We test this in two layers. A fast, in-process simulation runs real StripePacker implementations, each misbehaving one way, through the same validation a live PUT runs: a dropped unit, a duplicated unit, a hallucinated unit id, an out-of-bounds offset, a wrong bin count, and an under-reported capacity are all rejected before any shard is written. The last is a gap this testing found and we fixed: capacity was trusted uncritically, letting an under-report defeat the overhead check and silently truncate real data past the one existing correctness check. One more case is left failing on purpose: nothing yet checks a packer's output is deterministic across calls, unlike the placement-policy interface, which documents and tests this. Closing that gap, and sandboxing a packer's code rather than only its output, is the WebAssembly direction from the Introduction.

Case	Placement	Overhead	Write
Control (sane)	packed	0.00%	ok
A (lying header)	plain (fallback)	500.00%	ok
B (bad packer name)	plain (fallback)	n/a	ok
C (capacity-lying packer)	rejected	n/a	failed

TABLE II. Live cases, nine-node RS(6,3), 2% threshold. A/B retrigger the camera-ready round's original cases; C is this session's new finding.

One live case, on the same nine-node RS(6,3) cluster as Section III-A, confirms the fix: a bucket configured with the same capacity-lying packer is rejected outright (Table II, Case C), no object left reachable. Cases A and B retest the original camera-ready round's cases on this cluster. The dry-run preview's predicted overhead matches the real PUT's reported overhead exactly for both (0.00% control, 500.00% Case A).

IV. Conclusion & Future Work

WarpDial gives developers one cost-reporting interface for static, parametric, and imperative placement policies, the first object store to do so to our knowledge. On our nine-node cluster it reduces a selective TPC-H query's latency by ∼11.5× and Lance row-batch latency by up to ∼2.9×, falling to ∼1.9× at higher nprobe as the plain bucket's own range-GET path narrows the gap, with query results and Recall@10 unchanged throughout. The same mechanism keeps placement safe under adversarial and misconfigured input, detailed in Section III-B.

What remains is building the managed-service direction from Section III-B, with the overhead-threshold fallback as a shared backstop. Other directions include an SLA solver in place of a single threshold, a multi-VM deployment and comparison against another erasure-coded store, pool-wide failure-domain/fragmentation effects, and agent-driven policy configuration.

Acknowledgment

We used AI-agent assistance for language editing and phrasing. We thank all contributors to the WarpDrive source code.

References

[1] J. Lu et al., "Fusion: An Analytics Object Store Optimized for Query Pushdown," in Proc. ASPLOS, 2025.
[2] T. Chandrashekar, "WarpDrive: A Composable Storage Substrate for Disaggregated Data Systems," CDMS Workshop, VLDB 2026.
[3] M. Raasveldt and H. Mühleisen, "DuckDB: An Embeddable Analytical Database," in Proc. ACM SIGMOD, 2019.
[4] Transaction Processing Performance Council, "TPC Benchmark H (Decision Support)," Standard Specification.
[5] W. Pace et al., "Lance: Efficient Random Access in Columnar Storage through Adaptive Structural Encodings," arXiv:2504.15247, 2025.
[6] H. Jégou et al., "Product Quantization for Nearest Neighbor Search," IEEE Trans. Pattern Anal. Mach. Intell., vol. 33, 2011.